Universidad ORT Uruguay
Facultad de Ingeniería

Ciclos de feedback entre clientes y equipos de desarrollo en contextos de uso de IAG

Entregado como requisito para la obtención del título de Ingeniería en Sistemas

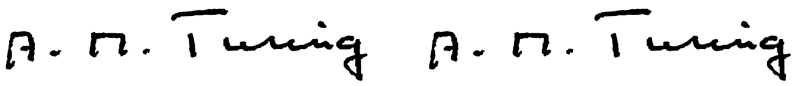
María Victoria Chappuis Adorni - 278311

Tutor: Dr. Ing. Daniel Calegari

2026

Yo, María Victoria Chappuis Adorni declaro que el trabajo que se presenta en esta obra es de mi propia mano. Puedo asegurar que:

- La obra fue producida en su totalidad mientras realizaba el Proyecto Final de Ingeniería en Sistemas;
- Cuando he consultado el trabajo publicado por otros, lo he atribuido con claridad;
- Cuando he citado obras de otros, he indicado las fuentes. Con excepción de estas citas, la obra es enteramente mía;
- En la obra, he acusado recibo de las ayudas recibidas;
- Cuando la obra se basa en trabajo realizado conjuntamente con otros, he explicado claramente qué fue contribuido por otros, y qué fue contribuido por mi;
- Ninguna parte de este trabajo ha sido publicada previamente a su entrega, salvo donde se han realizado las aclaraciones correspondientes.



Nombre1 Apellido1

Nombre2 Apellido2

1-3-2027

{DEDICATORIA}

{AGRADECIMIENTOS}

Abstract

El desarrollo de software depende del feedback del negocio: cada ciclo se cierra cuando el cliente confirma que lo especificado y lo construido responden a sus necesidades. La incorporación de herramientas de inteligencia artificial generativa (IAG) está reconfigurando esos ciclos, porque altera quién produce los artefactos del proceso, en qué momento están disponibles para ser revisados y quién está en condiciones de evaluarlos. Esta tesis analiza esa reconfiguración en las fases de implementación y validación, integrando la dimensión técnica y la validación de reglas y flujos de negocio. El estudio se apoya en una revisión bibliográfica sobre bases académicas y literatura gris, complementada con snowballing, en la que se revisaron aproximadamente 1750 resultados, y en una prueba de concepto exploratoria. Como contribución, se organiza la evidencia en categorías de técnicas según qué operación realiza la IA sobre el ciclo de feedback y se identifican las dimensiones que cada categoría altera: frecuencia, secuencia, granularidad y temporalidad. El análisis muestra que la literatura sostiene desplazamientos parciales del ciclo, no un cambio integral, concentrados en escenarios de asistencia bajo dominio humano, y que la interacción de validación del stakeholder sobre software en funcionamiento carece de estudios empíricos. La prueba de concepto, en curso, explora ese vacío mediante un asistente de sesiones de feedback que verifica la consistencia del feedback emitido contra las reglas de negocio.

Comentario: Borrador: actualizar cuando la prueba de concepto tenga resultados.

Palabras clave

ciclos de feedback; inteligencia artificial generativa (IAG); validación; aceptación

Índice

1. Introducción **8**
1.1. Contexto y motivación 8
1.2. Pregunta de investigación y objetivos 9
1.3. Estructura del documento 11

2. Antecedentes **12**
2.1. El ciclo de vida del software 12
2.1.1. La especificación de requisitos 13
2.2. Verificación y validación: el modelo en V 14
2.3. Ciclos de feedback: definición de trabajo 15
2.4. El estado actual del feedback: Scrum como ejemplo 16

3. El feedback mediado por IAG **18**
3.1. La inteligencia artificial generativa en el desarrollo de software . . . 18
3.2. Revisión bibliográfica 19
3.2.1. Proceso de búsqueda 19
3.2.2. Conjunto de trabajos resultante y su trazabilidad 21
3.3. Categorías de técnicas de feedback mediado por IAG 23
3.3.1. La IA emite el feedback: suplantación de stakeholders (*impersonation*) 23
3.3.2. La IA captura lo que se dijo: asistentes de reunión 25
3.3.3. La IA opera sobre el requisito 25
3.3.4. La IA construye un prototipo o aplicación 26
3.3.5. La IA valida contra reglas y flujos de negocio 27
3.3.6. El tramo técnico: generación de código 28
3.3.7. La instancia de aceptación 28
3.4. Qué dimensión del ciclo altera cada categoría 29
3.5. Las categorías en los escenarios de adopción S1 a S4 30
3.6. Aspectos de interés a profundizar 31

4. Prueba de concepto **33**
4.1. Definición . . . 33
4.2. Requisitos . . . 34
4.3. Diseño . . . 36
4.3.1. Arquitectura general . . . 36
4.3.2. Conectores de videollamada . . . 37
4.3.3. Procesamiento en el núcleo . . . 38
4.3.4. Base de conocimiento . . . 39
4.3.5. Conexión con el trabajo del equipo . . . 40
4.3.6. Recorte de la prueba de concepto . . . 40
4.4. Implementación . . . 41
4.5. Validación . . . 41

5. Conclusiones **42**

6. Referencias Bibliográficas **43**

7. Anexos **50**
7.1. Titulo Anexo 1 . . . 50
7.2. Titulo Anexo 2 . . . 51

1 Introducción

1.1 Contexto y motivación

El desarrollo de software se organiza en un ciclo de vida que comprende cuatro actividades fundamentales: la especificación; el diseño y la implementación; la validación; y la evolución [1]. Estas actividades no se ejecutan de forma aislada: el proceso avanza en la medida en que el negocio confirma que lo especificado y lo construido responden a sus necesidades. La validación, en particular, tiene como propósito asegurar que el sistema cumple con las expectativas del cliente, lo que exige instancias de intercambio entre quienes construyen el software y quienes lo encargan y lo usan.

Esta dependencia del *feedback* (retroalimentación) del negocio está institucionalizada en los modelos de proceso. El modelo en V hace explícita la correspondencia entre cada nivel de definición y su nivel de prueba, y ubica la prueba de aceptación, en la que el cliente valida el producto contra los requisitos, como contraparte directa de la especificación inicial [2]. Los enfoques ágiles llevan esa lógica a ciclos cortos: en Scrum, cada iteración (*sprint*) incluye una revisión en la que el equipo y los interesados inspeccionan el incremento y ajustan el rumbo [3]. En ambos casos, el feedback del negocio es el mecanismo que cierra cada ciclo y habilita el siguiente. Cuando lo que se valida involucra reglas y flujos de negocio, ese intercambio requiere además un entendimiento compartido de los procesos de la organización [4].

En los últimos años, la irrupción de la inteligencia artificial generativa (IAG) comenzó a transformar la forma en que se construye software. Las herramientas generativas se aplican hoy a lo largo de todo el proceso, tanto en la elicitación de requisitos como en la producción de código y de pruebas, y la literatura proyecta escenarios en los que la IAG deja atrás el papel de asistente ocasional para integrarse como un actor más del proceso de desarrollo [5]. Esta transformación abre una agenda de investigación amplia y todavía en formación [6].

La incorporación de IAG no altera solamente la productividad de tareas individuales: altera quién produce los artefactos del proceso y quién está en condiciones de evaluarlos. La evidencia disponible sugiere que la capacidad de generar se democratiza más rápido que la capacidad de validar: usuarios sin formación técnica logran producir artefactos con ayuda de herramientas generativas, pero evaluar, depurar y verificar lo generado sigue dependiendo de la experiencia [7]. A su vez, actores de negocio enfrentan dificultades para verificar por sí mismos análisis de datos generados por estas herramientas [8]. Los ciclos de feedback entre clientes y equipos de desarrollo, tal como los estructuran los modelos de proceso vigentes, quedan así sujetos a una reconfiguración cuyo alcance todavía no está caracterizado.

Esta tesis es un estudio exploratorio y descriptivo de esa reconfiguración, que integra la dimensión técnica y la validación de reglas y flujos de negocio. El estudio se apoya en una revisión bibliográfica del área y en una prueba de concepto que permite observar de forma directa dinámicas de validación temprana.

1.2 Pregunta de investigación y objetivos

La pregunta que guía la investigación es: £cómo se reconfiguran los ciclos de feedback entre clientes y equipos de desarrollo durante las etapas de implementación y validación de software cuando se incorporan herramientas de inteligencia artificial generativa? De ella se desprenden cinco subpreguntas:

- £Cómo participan los actores del negocio o clientes cuando se utilizan herramientas generativas en proyectos de software?
- £Cómo se transforman los roles dentro de los equipos de software en contextos mediados por herramientas generativas?
- £Cómo se caracterizan los procesos de feedback e iteración en entornos de desarrollo asistidos por IAG?
- £Qué efectos organizacionales o de coordinación se han reportado en equipos que utilizan herramientas de IAG?
- £Existe evidencia de modificaciones en los ciclos de desarrollo conocidos a partir del uso de asistentes generativos?

El objetivo general de la tesis, que operacionaliza esa pregunta, es analizar la reconfiguración de los ciclos de feedback en las etapas de implementación y validación de software cuando se incorporan herramientas de IAG.

Este objetivo general se desagrega en los siguientes objetivos específicos:

1. Caracterizar las dinámicas tradicionales de feedback técnico y de validación de procesos de negocio, como línea de base para el análisis de su reconfiguración.
2. Identificar cambios en la frecuencia, secuencia, granularidad y temporalidad de los intercambios de validación.
3. Examinar la transformación del rol del cliente, del analista funcional y del equipo técnico en contextos donde actores del negocio interactúan directamente con herramientas generativas.
4. Analizar el impacto de la IAG en la validación temprana de reglas de negocio, flujos de negocio y criterios de aceptación.
5. Describir, a partir de la literatura relevada, los efectos reportados en tiempos de ajuste, claridad de los requisitos, alineación entre negocio y tecnología, y coordinación del trabajo.
6. Desarrollar una prueba de concepto exploratoria de un prototipo basado en IAG orientado a usuarios no técnicos, que permita observar dinámicas de validación temprana de procesos.

El estudio no incluye trabajo de campo con equipos u organizaciones: los objetivos de caracterización y análisis se abordan sobre la base de la evidencia publicada, y la prueba de concepto aporta la instancia de observación directa. Quedan fuera del alcance el prototipado inicial, la ingeniería de requisitos inicial, el diseño temprano de experiencia de usuario y el mantenimiento posterior a la puesta en producción: el foco es el feedback durante el desarrollo iterativo y la aceptación sobre software ya construido. La prueba de concepto actúa en la sesión en la que el cliente revisa ese software, y confronta lo que dice con la documentación del proyecto. La definición del prototipo y de su forma de validación se presenta en el Capítulo 4.

1.3 Estructura del documento

El Capítulo 2 presenta el ciclo de vida del software, la especificación de requisitos, la verificación y validación según el modelo en V, la definición de trabajo de ciclo de feedback y el estado actual de esas dinámicas, con Scrum como ejemplo. El Capítulo 3 presenta primero la inteligencia artificial generativa y su incorporación al desarrollo de software, describe la revisión bibliográfica y luego presenta las categorías de técnicas identificadas, las dimensiones del ciclo que cada una altera, su ubicación en escenarios y los aspectos de interés que el relevamiento deja planteados. El Capítulo 4 presenta la prueba de concepto: su definición, los requisitos, el diseño, la implementación y la validación. Finalmente, el Capítulo 5 recoge las conclusiones y el trabajo futuro.

2 Antecedentes

Este capítulo presenta los conceptos sobre los que se apoya el resto del trabajo: cómo se organiza el proceso de desarrollo de software, cómo se documentan los requisitos, dónde se ubican la verificación y la validación, qué se entiende en esta tesis por ciclo de feedback y cómo ocurre ese intercambio en la práctica actual, tomando Scrum como ejemplo. Es un relevamiento del estado de la práctica tal como la describen los manuales de referencia y constituye la línea de base sobre la cual los capítulos siguientes analizan la reconfiguración estudiada.

2.1 El ciclo de vida del software

Un proceso de software es una secuencia de actividades que conduce a la producción de un producto de software, y cuatro actividades fundamentales son comunes a todos los procesos: la especificación, en la que clientes e ingenieros definen el software a producir y las restricciones de su operación; el diseño y la implementación; la validación, en la que se comprueba que el software es lo que el cliente requiere; y la evolución, en la que el software se modifica para reflejar los requisitos cambiantes del cliente y del mercado [1, secs. 1.1.1 y 2.2]. Estas actividades genéricas pueden organizarse de distintas formas y describirse a distintos niveles de detalle según el tipo de software, por lo que es posible razonar sobre ellas con independencia del modelo de proceso adoptado. No se desarrolla aquí cada actividad en detalle, salvo los documentos de la especificación, que se retoman en la Subsección 2.1.1; se remite a [1] para su tratamiento completo. La Figura 2.1 muestra cómo se organizan estas actividades en un proceso dirigido por plan, con cada nivel de prueba vinculado al nivel de definición que le corresponde.

De las cuatro actividades, dos involucran directamente al stakeholder. La especificación involucra por definición al cliente [1, cap. 4]. La validación comprueba lo construido contra lo que el cliente requiere, no solo contra el documento de especificación: la verificación y validación muestra que un sistema se ajusta a su

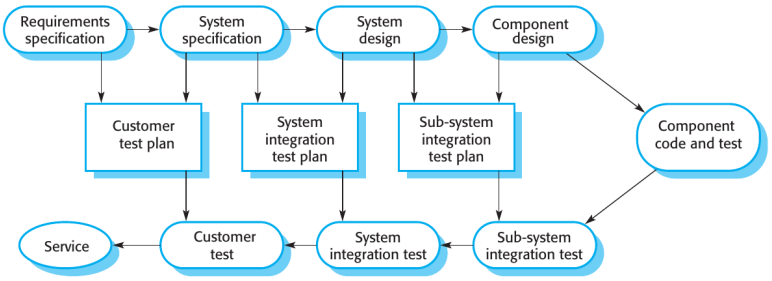


Figura 2.1: Fases de prueba en un proceso de software dirigido por plan. Fuente: [1, Fig. 2.7, sec. 2.2.3].

especificación y, a la vez, cumple las expectativas del cliente [1, sec. 2.2.3]. El diseño y la implementación son internos al equipo, y la evolución nace del stakeholder, pero se canaliza a través de las otras dos actividades: el cambio ingresa como requisito nuevo y se acepta validando lo modificado. Por ello, este trabajo se centra en la especificación y la validación. La implementación se entiende aquí como el período del proceso durante el cual ocurren esos intercambios; el objeto de estudio es el ciclo de feedback que la atraviesa, no la actividad de codificación en sí misma.

2.1.1 La especificación de requisitos

Especificar es escribir los requisitos de usuario y de sistema en un documento [1, sec. 4.4]. Esos documentos son la referencia de la validación, y la sesión de feedback también los alimenta: de ella surgen pedidos nuevos y decisiones que actualizan la especificación (Capítulo 4).

Sommerville no prescribe una forma única. Enumera cuatro notaciones para los requisitos de sistema: oraciones en lenguaje natural, lenguaje natural estructurado, notaciones gráficas como los casos de uso y especificaciones matemáticas [1, sec. 4.4]. Un documento de requisitos de software detallado no siempre hace falta: los enfoques ágiles suelen reemplazarlo por historias de usuario en tarjetas o pizarras [1, sec. 4.4.4]. La práctica añade documentos de requisitos de producto y artefactos informales como actas y glosarios, y conviven marcos como la norma ISO/IEC/IEEE 29148[1], la plantilla Volere[2] y Agile Modeling[3].

[1] `https://www.iso.org/standard/72089.html`

[2] `https://www.volere.org/templates/volere-requirements-specification-template/`

[3] `https://agilemodeling.com/essays/agileDocumentationBestPractices.htm`

Las guías del área sostienen lo mismo: SWEBOK (*Software Engineering Body of Knowledge*) señala que no hay respuestas estándar sobre la forma que deben adoptar los requisitos [9, cap. 1, tema 4], y el manual de IREB (*International Requirements Engineering Board*), que no existe una estructura documental universal [10, sec. 3.6]. La evidencia empírica es consistente con esta posición: una familia internacional de encuestas no identifica una técnica de documentación claramente predominante [11], y un estudio de entrevistas reporta que las especificaciones suelen apoyarse en plantillas o guías, pero rara vez siguen un estándar fijo [12].

2.2 Verificación y validación: el modelo en V

El modelo en V ordena las actividades del proceso en una V: la rama descendente son los niveles de definición; la ascendente, los de prueba; y la codificación queda en el vértice [2, cap. 2]. Su origen en la ingeniería de software se atribuye a [13]. La Figura 2.1 es la versión de este modelo que presenta Sommerville: el propio texto indica que, rotada, muestra la V [1, sec. 2.2.3]. La formulación clásica de la distinción es la de Boehm: la validación responde a la pregunta de si se está construyendo el producto correcto; la verificación, a la de si se lo está construyendo correctamente (citado en [1, cap. 8]).

Lo relevante para este trabajo no son los niveles en sí, sino las flechas horizontales: cada nivel de definición tiene su nivel de prueba correspondiente, y se prueba contra lo que ese nivel definió. El nivel superior, que empareja la especificación de requisitos con la prueba del cliente, es el único donde participa el stakeholder, y aparece en los dos extremos de la V. En el extremo izquierdo el stakeholder declara qué necesita; en el extremo derecho evalúa si lo construido lo satisface. Esta última es la instancia de aceptación: la prueba de aceptación (*acceptance testing*) es la forma de prueba con usuarios en la que el cliente prueba formalmente el sistema [1, cap. 8]. La flecha que une ambos extremos expresa que la aceptación se evalúa contra los requisitos que ese mismo stakeholder planteó. Los niveles inferiores, de integración de sistema, de subsistemas y de componentes, son internos al equipo.

El extremo derecho no es un evento puntual sino un proceso, que Sommerville descompone en seis etapas (Figura 2.2): (1) la definición de los criterios de aceptación, (2) la planificación, (3) la derivación de las pruebas, (4) su ejecución, (5) la negociación de los resultados y (6) la aceptación o el rechazo del sistema; las dos últimas constituyen un acuerdo entre el equipo de desarrollo y el cliente [1, sec. 8.4].

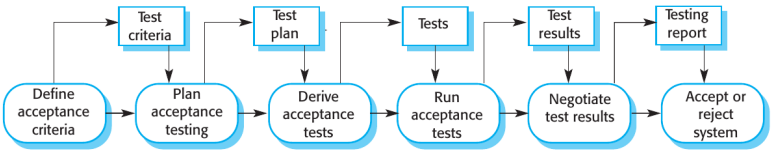


Figura 2.2: El proceso de prueba de aceptación. Fuente: [1, Fig. 8.11, sec. 8.4].

Tanto Pressman como Sommerville presentan el modelo en V como una variante del modelo en cascada. Aquí se lo toma como ilustración de la correspondencia entre definición y prueba, no como modelo de proceso a seguir. Cuántas veces se recorre la V es lo que cambia entre modelos de proceso, y es el punto de partida de la Sección 2.4.

2.3 Ciclos de feedback: definición de trabajo

En este trabajo se denomina ciclo de feedback al circuito formado por los dos extremos de la sección anterior: el stakeholder declara qué necesita, el equipo construye, y el mismo stakeholder indica si lo construido cumple lo planteado. El trabajo de diseño, implementación y evolución del equipo ocurre entre ambos puntos de contacto.

El término stakeholder se toma de Sommerville: incluye a cualquier persona afectada por el sistema y con un interés legítimo en él, desde los usuarios finales hasta gerentes y actores externos como reguladores [1, cap. 4]. Esa definición es amplia; esta tesis la acota a un representante del negocio que prueba el producto y opina de forma explícita y directa. El feedback indirecto, como el obtenido por telemetría u otros mecanismos automáticos de recolección, queda fuera del alcance. En este documento, cliente y stakeholder se usan como equivalentes. Junto al stakeholder interviene el analista funcional, rol clásico de validación de reglas de negocio, actividad cuyo anclaje conceptual está en la gestión de procesos de negocio [4, secs. 4.4 y 5.4.2].

El análisis de los capítulos siguientes describe el ciclo mediante cuatro dimensiones, que se adoptan como definiciones de trabajo de esta tesis. La frecuencia se refiere a la periodicidad del intercambio de feedback; la secuencia, al orden en que se dan las actividades del ciclo; la granularidad, al tamaño de la unidad sobre

la que se emite o se registra el feedback, desde un requisito completo hasta un fragmento de una funcionalidad; y la temporalidad, al momento del proceso en que llega el feedback respecto de la construcción.

2.4 El estado actual del feedback: Scrum como ejemplo

Los dos extremos del ciclo no ocurren una sola vez ni en un único orden: cuántas veces se recorre la V depende del modelo de proceso. En un proceso en cascada se recorre una vez, con los requisitos al inicio y la aceptación al final. En un proceso iterativo e incremental se recorre en cada iteración: cada entrega vuelve a abrir los dos puntos de contacto, y la aceptación de una iteración alimenta los requisitos de la siguiente. Para describir cómo ocurre hoy ese intercambio se toma Scrum como ejemplo, por ser una de las formas de trabajo más extendidas y por dar nombres establecidos a las actividades; el ejemplo ilustra el estado actual, no es la base del marco.

Scrum define una única instancia de contacto con el stakeholder, la *Sprint Review* (revisión del sprint), y la define como sesión de trabajo, no como reunión de aprobación: el equipo debe evitar limitarla a una presentación [3]. Dos precisiones de la Guía de Scrum importan aquí. Primero, Scrum separa la liberación de este evento: un incremento puede entregarse a los stakeholders antes del fin del sprint, y la Sprint Review nunca debe considerarse una condición para entregar valor [3]. El feedback puede entonces llegar antes del evento y por fuera de él. Segundo, el evento está acotado a un máximo de cuatro horas para un sprint de un mes, y a menos para sprints más cortos: el uso real del producto y la detección de desvíos no caben en esa instancia ni están definidos en ella. A la vez, la Guía de Scrum no define los términos *feedback*, *testing* y aceptación, y Scrum se declara deliberadamente incompleto [3]. Por eso la parte de ingeniería del intercambio (la liberación, la prueba de aceptación y la validación de reglas de negocio) se ancla fuera de Scrum, en [1] y [4].

El flujo, tal como ocurre hoy, es el siguiente. Luego de que el equipo de desarrollo libera un incremento, este se comunica al stakeholder. El stakeholder lo utiliza, lo acepta o no, y genera feedback que deriva en uno de dos resultados: incluir nuevos requisitos o modificar lo construido porque no lo valida contra los requisitos que él mismo definió; esa validación contra los requisitos propios es la flecha superior del modelo en V. Ese feedback llega por múltiples canales: por escrito, en llamadas o en instancias específicas. En particular, la Sprint Review es

una instancia definida para recorrer el sistema con el propio cliente y permite validar reglas de negocio, confirmando que lo construido se corresponde con la regla real; analizar la factibilidad de los cambios pedidos; y priorizar, decidiendo si ese feedback se incorpora o no a la próxima iteración.

La Figura 2.3 muestra el tramo completo en notación BPMN (*Business Process Model and Notation*): las actividades A1 a A5 sobre carriles según quién las realiza, con las que Scrum no define por completo en línea punteada. El tramo parte del desarrollo del incremento (A1), definido en Scrum. Del resto, Scrum define la Sprint Review (A2c) y el refinamiento de la lista ordenada de trabajo del producto (*Product Backlog*, A5); define solo parcialmente la incorporación del feedback al Product Backlog (A4) y el análisis de factibilidad técnica (A3b). El resto se ancla en los manuales: la liberación del incremento (A2a) y el uso y prueba de aceptación (A2b) en [1], y la validación de reglas de negocio (A3a) en [4]. El feedback del negocio se genera en A2b y A2c; el resto del tramo lo procesa. Son dos fuentes y dos momentos: el uso real del producto, que puede ser asincrónico y anterior al evento, y el evento mismo.

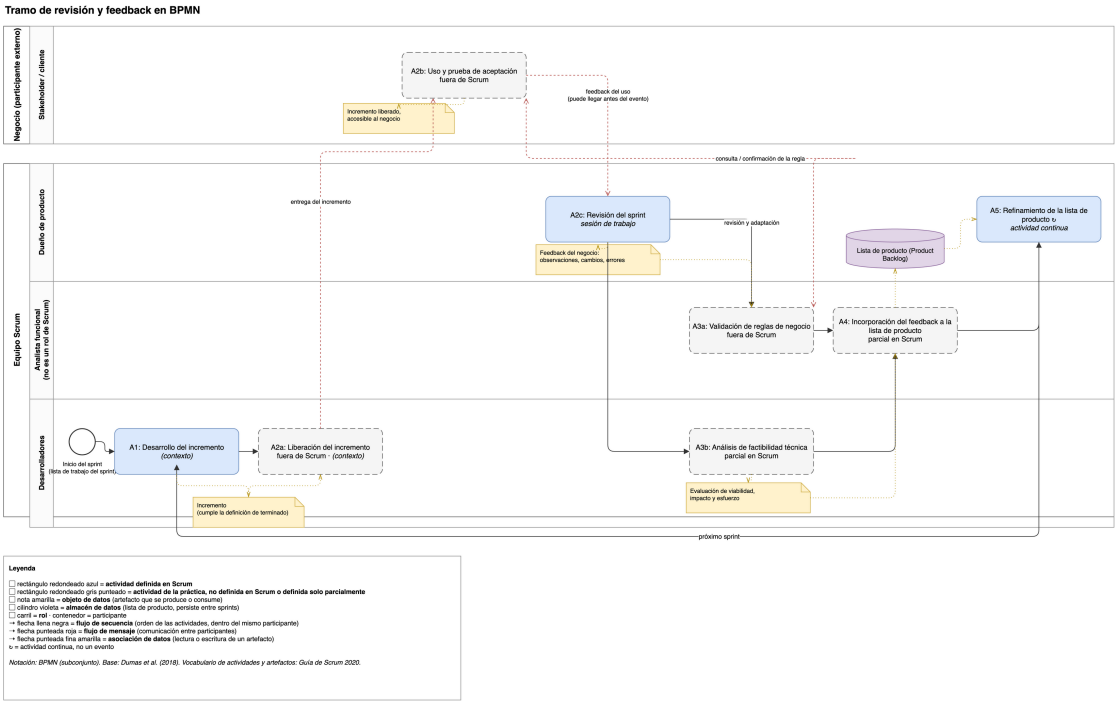


Figura 2.3: Tramo de revisión y feedback en Scrum, en BPMN. Actividades A1 a A5 sobre carriles según el rol que las realiza; en línea punteada, las que Scrum no define por completo. Elaboración propia sobre la notación de [4] y el vocabulario de [3].

3 El feedback mediado por IAG

Este capítulo constituye el eje del trabajo. Primero se introduce brevemente la inteligencia artificial generativa y su incorporación al desarrollo de software. Luego se describe la revisión bibliográfica realizada: el proceso de búsqueda y el conjunto de trabajos resultante. Sobre esa base se presentan las categorías de técnicas identificadas, leídas sobre la estructura de ciclos de feedback presentada en el Capítulo 2 y ordenadas según qué operación realiza la IA sobre el ciclo de feedback, con su evidencia, sus beneficios y sus riesgos y límites. A continuación se analiza qué dimensión del ciclo altera cada categoría y cómo se ubican las categorías en los escenarios de adopción de la literatura. El capítulo cierra con los aspectos de interés que el relevamiento deja planteados para profundizar.

3.1 La inteligencia artificial generativa en el desarrollo de software

La inteligencia artificial generativa (IAG) designa los sistemas capaces de producir contenido nuevo, en particular texto, código e imágenes, a partir de instrucciones expresadas en lenguaje natural. Su componente central son los modelos de lenguaje de gran escala (LLM, por *large language models*), modelos entrenados sobre grandes volúmenes de texto que generan salidas coherentes en respuesta a un pedido del usuario, sin que el usuario requiera conocimientos de programación [14] (en adelante se usa también IA cuando la distinción con otras técnicas no es relevante).

Desde 2022–2023, con la disponibilidad masiva de asistentes conversacionales y de herramientas de generación de código, estos sistemas se incorporaron al desarrollo de software y comenzaron a intervenir en tareas que van desde la elicitación y especificación de requisitos hasta la generación de código y de pruebas [5, 15]. La ingeniería de requisitos concentra buena parte de esa actividad: revisiones sistemáticas recientes relevan aplicaciones de LLM a las actividades de la ingeniería

de requisitos [16, 17, 18]. La literatura proyecta además escenarios donde la IAG deja de ser un asistente puntual para convertirse en participante del proceso de desarrollo [5], y formula una agenda de investigación amplia y todavía en formación sobre sus implicancias para la ingeniería de software [6].

Para esta tesis, lo relevante de esa transformación no es la productividad de las tareas individuales sino su efecto sobre el ciclo de feedback descrito en el Capítulo 2: la IAG modifica quién produce los artefactos sobre los que el negocio opina, en qué momento esos artefactos están disponibles para ser revisados y quién está en condiciones de evaluarlos. La Sección 3.2 describe cómo se relevaron los trabajos sobre el tema y las secciones siguientes presentan cómo esas técnicas operan sobre el ciclo. Las referencias de esta sección contextualizan el fenómeno y no integran el conjunto de trabajos descrito en la Sección 3.2, con la excepción del trabajo de Sauvola et al., que sí integra ese conjunto y se retoma en la Sección 3.5.

3.2 Revisión bibliográfica

Esta sección describe la revisión bibliográfica realizada para construir la base conceptual del trabajo. Se trató de una revisión bibliográfica y no de una revisión sistemática: el proceso partió de criterios de selección definidos y de lineamientos tomados de los estudios secundarios sistemáticos, pero no se sostuvo como protocolo cerrado, porque el objetivo no era garantizar la exhaustividad ni la repetibilidad sino relevar el estado del tema, y porque el foco del trabajo se fue ajustando durante el proceso, de modo que los criterios aplicados al inicio no coinciden exactamente con los que resultaron pertinentes al final. El proceso queda de todos modos documentado. La Sección 3.2.1 presenta el proceso de búsqueda y la Sección 3.2.2, el conjunto de trabajos resultante y su trazabilidad.

3.2.1 Proceso de búsqueda

Se utilizó la siguiente cadena de búsqueda (*search string*), consolidada tras probar variantes equivalentes:

```
"generative AI" AND ("business users" OR clients OR stakeholders)
AND "software development"
```

con un filtro temporal desde 2023, tomando como referencia el lanzamiento de ChatGPT a fines de 2022, hasta marzo de 2026, momento de ejecución de las búsquedas. La cadena se ejecutó sobre cinco bases académicas: ACM Digital Library, IEEE Xplore, ScienceDirect, SpringerLink y Wiley Online Library.

El filtrado aplicó criterios de selección. Las bases se consultaron con el filtro de revisión por pares activado, y se aceptaron trabajos en inglés o español, publicados como artículos de revista, artículos de conferencia o revisiones, que cumplieran simultáneamente tres criterios temáticos: abordar el uso de IAG en el contexto del desarrollo de software, involucrar stakeholders, clientes o usuarios no técnicos, y tratar el feedback, la validación o la comunicación entre equipos. Se rechazaron los trabajos en otros idiomas y los libros, tesis, reportes técnicos, editoriales y opiniones sin revisión por pares.

Con esos criterios, los resultados se filtraron en fases sucesivas: una primera clasificación por título en tres niveles de relevancia (alta pertinencia, a revisar, descarte), una segunda fase en la que se leyeron los resúmenes de los trabajos no descartados, y una resolución final de los casos dudosos revisada con el tutor. De ese filtrado quedó un primer conjunto de diez trabajos. Sobre la mayor parte de ellos se realizó *snowballing* en ambas direcciones: hacia atrás, relevando sus referencias (cerca de 500 entradas), y hacia adelante, relevando los trabajos que los citan (cerca de 160).

La búsqueda se complementó con literatura gris, usando la misma cadena adaptada a cada buscador sobre Google, arXiv, SSRN y Zenodo, y con búsquedas posteriores más específicas: por temas (feedback, roles, validación de negocio, efectos percibidos, usuarios no técnicos) y un relevamiento de productos comerciales que operan sobre el ciclo de feedback. Posteriormente se realizó una búsqueda complementaria y dirigida, con vocabulario de aceptación, orientada a un tramo específico del ciclo; sus resultados se reportan por separado.

En total se revisaron aproximadamente 1750 resultados entre las búsquedas principales. De ellos, solo 19 quedaron clasificados como relevantes, cerca del 1 %: la gran mayoría de lo recuperado cruza la IAG con el desarrollo de software, pero no trata el feedback entre los actores del negocio y el equipo de desarrollo, que es el objeto de este trabajo. De esos 19 trabajos, 12 se citan en las categorías del marco que se presenta en la Sección 3.3; los 7 restantes quedaron relevados pero sus aportes no llegaron a integrarse al marco.

3.2.2 Conjunto de trabajos resultante y su trazabilidad

El conjunto sobre el que se construye el marco reúne 18 trabajos, además de los 8 productos del relevamiento de productos comerciales. Los 18 tienen origen registrado: 7 surgen directamente del filtrado de las búsquedas en bases académicas y del snowballing, 5 de la búsqueda de literatura gris, 3 de la búsqueda complementaria con vocabulario de aceptación, 2 fueron aportados por el tutor y 1 proviene de la exploración inicial del tema, previa a las búsquedas. Los 12 trabajos citados que provienen de las búsquedas, mencionados en la Sección 3.2.1, son los 7 del filtrado y el snowballing más los 5 de la literatura gris. El conjunto se compone de un corpus de trabajos con revisión por pares y de literatura gris; esa distinción se retoma en la Sección 3.3 al declarar la fuente de evidencia de cada categoría. La Tabla 3.1 lista los 18 trabajos con su año, su origen y su tipo de evidencia. El origen y el tipo son independientes: los trabajos de Alabsi et al. y de Fawzy et al., recuperados por el snowballing y por la búsqueda de aceptación respectivamente, circulan como *preprints* (prepublicaciones) y cuentan como literatura gris al declarar la evidencia.

Trabajo	**Año**	**Origen**	**Evidencia**
Abbasi et al. [19]	2025	Búsquedas y snowballing	Revisada por pares
Cabrero-Daniel et al. [20]	2024	Búsquedas y snowballing	Revisada por pares
Kretzer et al. [21]	2025	Búsquedas y snowballing	Revisada por pares
Lindenberg et al. [22]	2025	Búsquedas y snowballing	Revisada por pares
Mircea et al. [23]	2026	Búsquedas y snowballing	Revisada por pares
Robinson et al. [24]	2025	Búsquedas y snowballing	Revisada por pares
Alabsi et al. [25]	2026	Búsquedas y snowballing	Gris (preprint)
Busany et al. [26]	2024	Literatura gris	Gris (preprint)
Damian et al. [27]	2025	Literatura gris	Gris (preprint)
Geyer et al. [28]	2025	Literatura gris	Gris (preprint)
Torun et al. [29]	2025	Literatura gris	Gris (preprint)
Violino [30]	2026	Literatura gris	Gris (prensa profesional)
Virk y Liu [8]	2025	Búsqueda de aceptación	Revisada por pares
Sharma et al. [31]	2026	Búsqueda de aceptación	Revisada por pares
Fawzy et al. [7]	2026	Búsqueda de aceptación	Gris (preprint)
Cornide-Reyes et al. [32]	2025	Aporte del tutor	Revisada por pares
Sauvola et al. [5]	2024	Aporte del tutor	Revisada por pares
Pirozzi [33]	2024	Exploración inicial	Gris (prensa profesional)

Tabla 3.1: Trabajos que integran el marco: origen y tipo de evidencia.

Tres de los trabajos incorporados por literatura gris [28, 29, 26] habían sido

recuperados por las búsquedas en bases académicas o por el snowballing y descartados en las fases de clasificación, y reingresaron al conjunto de trabajos del marco en la búsqueda de literatura gris, cuando el foco del trabajo ya se había ajustado. Estos casos de descarte y reingreso son el motivo principal por el que el proceso se presenta como revisión bibliográfica: los criterios aplicados al inicio no coinciden exactamente con los que resultaron pertinentes al final, reflejo del ajuste de foco ya señalado al inicio de la sección.

Para verificar que los descartes no ocultaran material pertinente, una vez consolidado el marco se recodificaron los trabajos descartados contra sus categorías: se reexaminaron los 1066 registros únicos del proceso de clasificación, de los cuales 389 combinan vocabulario de feedback con IAG. La recodificación no requirió crear categorías nuevas: los más de veinte candidatos que surgieron de ella se ubican en las categorías existentes, en su mayoría en las que describen operaciones que la IAG ya cubre, y los restantes son casos límite que realizan la operación en otro dominio o que generan el artefacto de aceptación. Ninguno estudia la validación del stakeholder sobre software en funcionamiento, de modo que el aspecto que la Sección 3.6 señala como principal interés a profundizar tampoco aparece cubierto en el material descartado.

La revisión tiene límites de alcance que conviene dejar establecidos. Las cadenas de búsqueda se formularon en inglés. El filtrado lo realizó una sola revisora, con los casos dudosos resueltos en consulta con el tutor. La cobertura de los resultados recuperados fue parcial en dos bases (se cargaron 192 de 313 resultados de ScienceDirect y 100 de 136 de Wiley), Web of Science quedó fuera por falta de acceso a la búsqueda por documentos, y en la literatura gris se relevaron los primeros 150 resultados de cada fuente, aproximadamente. La literatura del tema es además muy reciente e incluye una proporción alta de preprints sin revisión por pares, por lo que parte de los hallazgos citados puede no sostenerse en las versiones publicadas.

El resultado de la revisión no es una lista de trabajos sino la estructura que organiza sus aportes: las categorías de técnicas de feedback mediado por IAG y las dimensiones del ciclo que cada una altera, que se presentan en las secciones siguientes.

3.3 Categorías de técnicas de feedback mediado por IAG

Los trabajos relevados se agruparon en categorías según un criterio único: qué operación realiza la IA sobre el ciclo de feedback entre el negocio y el equipo de desarrollo. Resultaron seis categorías de técnicas, más una séptima que corresponde a la instancia de aceptación, donde el relevamiento arrojó un resultado de otra naturaleza: la ausencia de estudios encontrados sobre la interacción de validación del stakeholder. Cada categoría se presenta con su descripción, los trabajos que la respaldan y sus beneficios, riesgos y límites verificados (constatados en los propios estudios). La Tabla 3.2 resume las seis primeras.

La evidencia que sostiene cada categoría no es homogénea y se declara en cada caso. Proviene de tres fuentes: el corpus de la revisión (trabajos con revisión por pares, Sección 3.2), literatura gris (preprints y prensa profesional, sin revisión por pares) y evidencia de mercado (productos comerciales relevados como estado de la práctica). La evidencia de mercado corresponde a información publicada por los propios proveedores y no a evaluaciones independientes. Las categorías responden a los objetivos específicos planteados en el Capítulo 1; en las categorías de técnicas se indica a qué objetivo responde cada una.

3.3.1 La IA emite el feedback: suplantación de stakeholders (*impersonation*)

En esta categoría, la IA asume el rol del stakeholder: responde consultas del equipo como lo haría un representante del negocio, de modo que hay un interlocutor disponible en cualquier momento del desarrollo, sin coordinar agendas. El efecto sobre el ciclo es doble: el stakeholder real deja de ser un cuello de botella y, a la vez, cambia de rol, porque pasa de emisor del feedback a validador de lo que la IA respondió en su nombre. La categoría responde al objetivo específico sobre la transformación de roles (objetivo 3 del Capítulo 1).

La evidencia disponible es literatura gris, sin respaldo del corpus revisado por pares. Pirozzi propone el concepto de *AI-stakeholder* desde la prensa profesional de gestión de proyectos [33], y la propuesta REConnect explora la participación de stakeholders mediada por IA en ingeniería de requisitos [27].

El riesgo central es que la IA no es el stakeholder real: el feedback puede

Categoría	**Evidencia**	**Beneficio**	**Riesgos y límites**
Emite el feedback (suplantación de stakeholders)	Literatura gris [33, 27]; sin respaldo del corpus	Interlocutor disponible en cualquier momento; el stakeholder real pasa de emisor a validador	El feedback puede resultar plausible y ser incorrecto; el humano debe quedar como curador de las salidas
Captura lo que se dijo (asistentes de reunión)	Corpus [20]	Lo dicho queda como registro accionable en vez de perderse	Pérdidas en el resumen; solo reuniones internas, sin clientes
Opera sobre el requisito	Corpus [23, 19]; gris [28, 29]; mercado [34, 35]	El enunciado llega al equipo más claro y completo sin dejar de ser del stakeholder	Sustitución de la intención original; estudios sin actores de negocio ni software construido
Construye un prototipo o aplicación	Corpus [21, 24]; gris [26, 25, 30]; mercado [36, 37]	Adelanta el momento en que el stakeholder puede reaccionar ante algo concreto	El prototipo puede inducir la respuesta; costo en experiencia de usuario [32]; evidencia empírica débil
Valida contra reglas y flujos de negocio	Corpus [22], previo a validar	Previsto: validación temprana y continua contra las reglas del negocio	Sin trabajos de validación propiamente dicha; la regla mal capturada queda como criterio fijo
Tramo técnico (generación de código)	Mercado [38, 39, 40, 41]	No se evalúa (fuera del alcance)	Se registra solo como contexto

Tabla 3.2: Categorías de técnicas de feedback mediado por IAG: evidencia, beneficio, riesgos y límites.

resultar plausible y ser incorrecto. La propia propuesta REConnect pide que el humano quede como *"curator of AI outputs"*, lo que constituye un límite a la sustitución del stakeholder, no a la técnica en sí [27].

3.3.2 La IA captura lo que se dijo: asistentes de reunión

En esta categoría, la IA registra y estructura lo que se dijo en una reunión: produce resúmenes y elementos accionables, como tareas (*tickets*), a partir de la conversación. El beneficio es que lo dicho queda como registro en vez de perderse. La categoría responde a los objetivos sobre cambios en los intercambios de validación y sobre efectos reportados (objetivos 2 y 5).

El respaldo proviene del corpus: Cabrero-Daniel et al. estudian asistentes de reunión basados en LLM personalizados para equipos ágiles [20]. En ese estudio, el asistente además detectó riesgos de sobrecompromiso e impedimentos que el propio equipo no veía.

Los límites verificados acotan el alcance de la categoría para esta tesis: el estudio cubre solo reuniones internas del equipo, sin instancias con clientes, y el asistente no transcribe de forma autónoma, ya que trabaja sobre planillas cargadas manualmente. Por eso la categoría aporta contexto al marco. Los riesgos señalados son las pérdidas en el resumen y que la presencia del asistente cambie lo que las personas dicen.

3.3.3 La IA opera sobre el requisito

En esta categoría, la IA opera sobre el enunciado del requisito: lo reformula, juzga su calidad o deriva entradas dispersas hacia el artefacto correspondiente. El beneficio es que el enunciado del stakeholder llega al equipo más claro y completo sin dejar de ser suyo. Responde a los objetivos 2 y 5.

Es la categoría con evidencia empírica más sólida del relevamiento. Mircea et al. la miden con 26 participantes y 130 pares de requisitos: todas las dimensiones de calidad evaluadas mejoran con la revisión del LLM, con mejoras estadísticamente significativas; el 43 % de las revisiones revela aspectos que el autor no había expresado y consideraba importantes, y solo el 5 % introduce errores [23]. El esquema de trabajo es que el LLM reformula y el humano aprueba, encuadre que los autores denominan *AI-in-the-Loop* y que oponen explícitamente a tratar al LLM

como productor autónomo. Abbasi et al. proponen un marco de sinergia humano-IA para ingeniería de requisitos [19]. En la literatura gris, Geyer et al. estudian la evaluación de calidad de épicas (*epics*, requisitos de gran alcance) con IAG [28] y Torun et al. analizan el rol de la IAG en el seguimiento de defectos [29]. Como evidencia de mercado, Kraftful convierte feedback de llamadas, soporte y reseñas en tickets [34] y Versive transforma entrevistas con clientes en reportes con citas [35].

Los límites verificados matizan el alcance. En el estudio de Mircea et al. los participantes son técnicos, sin actores de negocio, y se trata de elicitación inicial sin software construido en ningún punto. El marco de Abbasi et al. no tiene evaluación empírica y su usuario es el ingeniero, no el stakeholder. El trabajo de Geyer et al. fue evaluado por 17 gerentes de producto, no por clientes. El de Torun et al. es el único de la categoría que trabaja sobre software ya construido. Los riesgos identificados son que el LLM sustituya la intención original en vez de articularla, y que el criterio del evaluador automático reemplace al del negocio sin que nadie lo note.

3.3.4 La IA construye un prototipo o aplicación

En esta categoría, la IA materializa tempranamente el requisito (en adelante, materialización temprana): produce un prototipo o una aplicación que el stakeholder puede ver o usar. El relevamiento distingue dos variantes según quién opera la herramienta: (a) la opera el equipo de desarrollo, y (b) la opera directamente el usuario de negocio, sin intermediario técnico. La categoría responde a los objetivos 2, 3 y 4.

El beneficio es que adelanta el momento en que el stakeholder puede reaccionar: opina sobre algo concreto y usable, no sobre una descripción, lo que revela requisitos implícitos. Robinson et al. lo formulan así: *"seeing a tangible product can unearth elements of the requirements that were assumed subconsciously but not articulated"* [24].

En la variante operada por el equipo, el respaldo del corpus es el asistente de Kretzer et al., que cierra el ciclo entre historias de usuario y prototipos de interfaz [21]; en la literatura gris se suman la generación de requisitos de inteligencia de negocio [26] y la elicitación y prototipado asistidos a partir de bocetos [25]. En la variante operada por el usuario de negocio, el corpus aporta la visión de Robinson et al. sobre ingeniería de software de usuario final [24], y la evidencia de mercado muestra productos activos: Emergent lleva una intención en lenguaje natural a una

aplicación desplegada para usuarios no técnicos [36], y PM Agent convierte una llamada en una especificación con criterios de aceptación y tickets [37]. La prensa profesional registra el mismo movimiento en las organizaciones: los directores de sistemas (*CIO*) permiten a usuarios de negocio construir sus propias aplicaciones con asistentes de IA, con la gobernanza del área técnica como resguardo [30].

Los límites verificados son importantes. El estudio de Kretzer et al. es intra-equipo, sin participación de clientes. El de Alabsi et al. no tuvo participantes: el rol del stakeholder fue simulado y la generación se hizo de una sola vez, por lo que su afirmación de que habilita validación más temprana es inferencial, no medida. El planteo de Robinson et al. es una visión sin evidencia empírica. Entre los riesgos, el prototipo puede inducir la respuesta del stakeholder en vez de revelarla, y la validación rápida sacrifica experiencia de usuario [32]. Además, la reorientación de los productos Tusk y Sweep, discutida en la categoría del tramo técnico (Sección 3.3.6), sugiere los límites del modo totalmente autónomo.

3.3.5 La IA valida contra reglas y flujos de negocio

Esta categoría corresponde a parte del planteo de esta tesis: validar de forma temprana y continua que lo construido respeta las reglas y los flujos del negocio, sin esperar a la instancia de aceptación. Responde al objetivo 4.

Es una categoría prevista antes que respaldada: el relevamiento no encontró trabajos de validación propiamente dicha. El único trabajo del corpus cercano, de Lindenberg et al., descubre procesos de negocio mediante agentes basados en LLM, es decir, opera antes de la validación, y su evaluación es técnica, sin participantes humanos; el trabajo se relevó a partir de su resumen publicado, por falta de acceso al texto completo [22]. La base conceptual de la categoría es la validación de reglas de negocio y la validación semántica de modelos de proceso tal como las describe la gestión de procesos de negocio [4], actividades cuya base conceptual se citó en el Capítulo 2.

Los riesgos, también previstos, son dos: que la regla capturada sea incorrecta y todo se valide contra esa versión sin que el negocio lo note, y que el criterio del negocio quede codificado y deje de discutirse.

3.3.6 El tramo técnico: generación de código

La generación de código con validación humana constituye el tramo técnico del ciclo y queda fuera del alcance de esta tesis, que no juzga la generación de código. Se registra únicamente la evidencia de mercado como contexto: Devin y Codegen convierten tickets en solicitudes de integración (*pull requests*) con aprobación del desarrollador [38, 39], mientras que Tusk y Sweep, que apuntaban al mismo tramo de ticket a *pull request* con la ambición de operarlo de forma autónoma, abandonaron ese modo y reorientaron su producto [40, 41]. Esa reorientación constituye contraevidencia de mercado sobre la viabilidad del modo totalmente autónomo.

3.3.7 La instancia de aceptación

La séptima categoría corresponde a la instancia de aceptación: el momento en que el stakeholder usa el software construido e indica si es lo que pidió. Como complemento de la búsqueda principal, se revisó la literatura con el vocabulario propio de ese tramo (pruebas de aceptación, criterios de aceptación (*acceptance criteria*), UAT (*user acceptance testing*) y Gherkin, el lenguaje de escenarios de BDD (*behavior-driven development*)), que un trabajo de esa área puede usar sin nombrar a la IAG ni a los stakeholders (Sección 3.2.1).

Pese a estar dirigida específicamente a ese tramo del ciclo, la búsqueda arrojó trabajos que apuntan a generar el artefacto de aceptación, es decir, pruebas, criterios o escenarios Gherkin, con el desarrollador, el encargado de pruebas o el dueño de producto (*Product Owner*) como usuario, y que, donde abordan la aceptación, buscan automatizarla. No se encontraron estudios de la interacción de validación del stakeholder sobre software en funcionamiento.

El proceso de aceptación descrito por Sommerville permite ubicar con precisión ese vacío. El manual lo descompone en seis etapas, reproducidas en la Figura 2.2: definir los criterios, planificar, derivar las pruebas, ejecutarlas, negociar los resultados y aceptar o rechazar el sistema [1]. Lo relevado se concentra en definir criterios, derivar pruebas y ejecutarlas (etapas 1, 3 y 4); sobre las restantes no se encontraron trabajos, y en particular sobre las dos cuyo producto es un acuerdo entre personas: la negociación de resultados y la aceptación o el rechazo (etapas 5 y 6). El propio Sommerville anticipa por qué esas etapas son irreductiblemente interpersonales: *"the developer and the customer have to negotiate to decide if the system is good enough to be used"* [1, sec. 8.4]. En ese tramo se ubica esta tesis. Lang y Fischer leen la escasez de trabajos sobre validación como señal de que esa

actividad se presta menos a la automatización con IA [18]; esa lectura no alcanza a este trabajo, que no propone automatizar la validación sino estudiar cómo se reconfigura la interacción de validación cuando la IAG interviene en el ciclo.

Este resultado importa porque la evidencia disponible sugiere una asimetría entre la capacidad de generar y la de validar, que se retoma en la Sección 3.6. Esa evidencia proviene de los tres trabajos incorporados por la búsqueda complementaria de aceptación que contabiliza la Sección 3.2.2. Virk y Liu encuentran que profesionales de negocio no logran verificar de forma confiable salidas generadas por IA, incluso cuando se los instruye para buscar errores [8]. Cabe precisar que lo validado en ese trabajo fue un análisis de datos y no software en funcionamiento; en esa diferencia queda el margen que esta tesis explora. Fawzy et al. reportan lo mismo en no programadores que construyen software con IA: la capacidad de evaluar, depurar y verificar sigue dependiendo de la experiencia [7]. Sharma et al. sistematizan cuatro barreras al feedback de calidad (la base común o *common ground*, la verificabilidad, la comunicación y la informatividad) y diseñan apoyos que lo mejoran [31]; ese apoyo es un insumo directo para la prueba de concepto del Capítulo 4.

3.4 Qué dimensión del ciclo altera cada categoría

El objetivo 2 pide identificar cambios en la frecuencia, la secuencia, la granularidad y la temporalidad de los intercambios de validación. Aplicada esa matriz a las categorías anteriores, el resultado es el siguiente. La categoría que emite el feedback altera la frecuencia y la temporalidad, porque sustituye al emisor: el intercambio puede ocurrir en cualquier momento y tantas veces como se necesite. La categoría que captura lo que se dijo altera la granularidad del registro. La que opera sobre el requisito altera la granularidad y la secuencia. La que construye un prototipo o aplicación altera la secuencia y la temporalidad, porque adelanta el momento en que hay algo visible. Para la categoría que valida contra reglas y flujos de negocio no hay evidencia sobre la cual afirmar alteraciones, y el tramo técnico queda fuera de la matriz por estar fuera del alcance declarado. La Tabla 3.3 resume el mapeo.

Ninguna categoría altera las cuatro dimensiones a la vez. Lo que la literatura relevada sostiene son desplazamientos parciales del ciclo de feedback, no un cambio integral.

Categoría	Frecuencia	Secuencia	Granularidad	Temporalidad
Emite el feedback	×			×
Captura lo que se dijo			×	
Opera sobre el requisito		×	×	
Construye un prototipo o aplicación		×		×
Valida contra reglas y flujos de negocio		sin evidencia		

Tabla 3.3: Dimensiones del ciclo de feedback alteradas por cada categoría.

3.5 Las categorías en los escenarios de adopción S1 a S4

Las categorías presentadas muestran un mismo movimiento de fondo: actividades que hoy realiza una persona empiezan a ser asumidas por la IA. Sauvola et al. proponen una escala de cuatro escenarios de adopción de IAG en el desarrollo de software, de S1 a S4, según el grado en que la IA asume el trabajo [5]. Esa escala permite ordenar las categorías por cuánto desplazan al humano, y no solo sobre qué actividad operan. La escala va de S1, el proceso de desarrollo tradicional en el que los humanos ocupan todos los roles, a S4, donde la IA gestiona las operaciones de desarrollo y el rol humano se concentra en la supervisión y el control, pasando por la asistencia de la IA bajo dominio humano (S2) y la asunción de roles seleccionados por la IA (S3). La Figura 3.1 presenta el mapeo, elaborado a partir de las Tablas 1 y 2 de ese trabajo.

Del mapeo se lee que lo relevado se concentra en S2 y alcanza S3 en dos puntos; nada llega a S4. Es un resultado, no una omisión: en ninguna categoría el humano sale del circuito. Los dos puntos que alcanzan S3 lo hacen además de maneras incompatibles entre sí: o la IA sustituye a quien emite el feedback (la categoría de suplantación de stakeholders), o el feedback lo emite un actor de negocio que ahora también construye (la variante de materialización operada por el usuario de negocio).

El mapeo tiene dos salvedades. Primero, los escenarios de Sauvola et al. describen la organización del trabajo de desarrollo en general, no específicamente el ciclo de feedback con el stakeholder, por lo que la ubicación de las categorías es una lectura propia de este trabajo. Segundo, varias categorías tienen variantes en escenarios distintos, en particular la de materialización temprana, de modo que lo ubicado en la figura no siempre es la categoría entera.

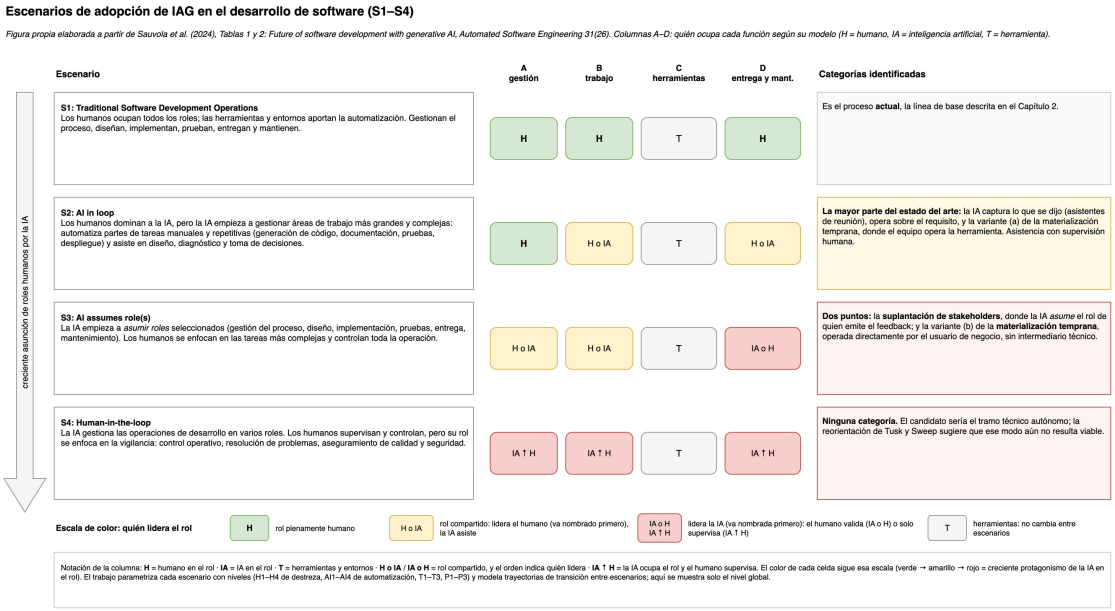


Figura 3.1: Las categorías de técnicas sobre los cuatro escenarios de adopción de IAG en el desarrollo de software (S1 a S4). Elaboración propia a partir de las Tablas 1 y 2 de Sauvola et al. [5].

3.6 Aspectos de interés a profundizar

El relevamiento deja planteados aspectos que la literatura relevada no cubre y que constituyen líneas de interés para profundizar. Se presentan como contexto para orientar la experimentación de esta tesis, no como una propuesta de trabajo.

El aspecto principal es la instancia de aceptación con el stakeholder real sobre software en funcionamiento. Como se describió en la categoría correspondiente, la literatura relevada trabaja sobre la generación del artefacto de aceptación y sobre IA que procesa feedback ya emitido; no se encontró trabajo empírico sobre cómo se reconfigura la interacción de validación en sí, el momento en que el stakeholder usa lo construido e indica si es lo que pidió.

A ese aspecto central se suman tres necesidades sin trabajo asociado en lo relevado:

- La validación de reglas y flujos de negocio con IAG (objetivo 4). La categoría correspondiente no tiene trabajos de validación propiamente dicha: esa parte del planteo de esta tesis no encuentra respaldo en la literatura relevada, lo que la convierte en terreno a explorar.

- El stakeholder como validador de lo que la IA produjo. La evidencia disponible sugiere que la capacidad de generar se democratiza más rápido que la de validar: los profesionales de negocio no logran verificar de forma confiable salidas generadas [8], y aun en quienes construyen con IA la capacidad de evaluar lo generado sigue atada a la experiencia [7]. No se encontraron estudios sobre apoyos para que actores de negocio verifiquen salidas sobre software en funcionamiento; los apoyos de Sharma et al., citados en la instancia de aceptación, son lo más cercano y están formulados para otro contexto, como recoge el punto siguiente.
- La calidad del feedback que el stakeholder logra emitir. Las cuatro barreras identificadas por Sharma et al. están formuladas para agentes conversacionales, no para el ciclo entre el cliente y el equipo de desarrollo [31]. Trasladarlas a ese ciclo es tarea pendiente, y es el punto de partida conceptual de la prueba de concepto del Capítulo 4.

4 Prueba de concepto

Este capítulo presenta un producto basado en IAG para las sesiones de feedback entre el cliente y el equipo de desarrollo, y su prueba de concepto.

4.1 Definición

El producto tiene un núcleo que confronta lo dicho con las reglas de negocio y los requisitos documentados, y le responde al cliente citando dónde contradice lo acordado. Se usa directamente en la aplicación web (en adelante, la web), que además muestra el detalle de cada veredicto, o mediante conectores a otras plataformas. El primero es un *bot*, un programa que entra a la reunión de Google Meet como participante y responde en ella.

El criterio rector es que el actor de negocio quede afectado: el resultado de la validación vuelve a él, no al equipo, y cambia lo que hace o entiende.

La prueba de concepto materializa el objetivo de desarrollar un prototipo basado en IAG para usuarios no técnicos (Capítulo 1). También permite observar cambios en la granularidad y la temporalidad del intercambio, porque la respuesta llega en la sesión; la transformación de los roles, porque la IAG media el control de consistencia del analista funcional y el actor de negocio interactúa directamente con la herramienta; y la validación temprana de reglas de negocio, porque cada contradicción se plantea en la sesión.

El producto se diseña sobre el traslado de las cuatro barreras al feedback de calidad de Sharma et al. [31] al ciclo entre el cliente y el equipo de desarrollo (Sección 3.6): *base común*, la documentación como referencia compartida por ambas partes (RF1); *verificabilidad*, la cita comprobada (RF2, RNF2); *comunicación*,

la pregunta al cliente (RF4); e *informatividad*, qué se contradice y dónde, para que la devolución sea accionable (RF1, RF2). RF y RNF designan los requisitos funcionales y no funcionales de la Sección 4.2.

4.2 Requisitos

Esta sección especifica el producto al que evolucionaría la prueba de concepto si se desarrollara profesionalmente. Cada requisito describe comportamiento externo; las decisiones técnicas figuran en la Sección 4.3 [1, sec. 4.4]. La columna Alcance toma cuatro valores: *PoC* (*proof of concept*, prueba de concepto), construido; *Parcial*, construido en parte, con su límite entre paréntesis; *A explorar*, candidato a sumarse; *Futuro*, fuera de esta tesis. Construido no significa medido ni validado con usuarios (Sección 4.5). Los requisitos funcionales van del núcleo a los conectores e integraciones y, en cada grupo, de mayor a menor importancia.

ID	El producto debe	Alcance
Núcleo: validación contra la documentación		
RF1	Dar un veredicto por enunciado: contradice lo acordado, coincide, no está contemplado o no es accionable.	PoC
RF2	Justificar el veredicto con una cita comprobada en el documento.	PoC
RF3	Reconocer pedidos, reglas y afirmaciones, y descartar el resto.	PoC
RF4	Ante una contradicción o un enunciado ambiguo, preguntarle al cliente; ante una contradicción, pedirle además que decida.	PoC
RF5	Validar contra toda forma de documentar (historias de usuario, casos de uso, actas, glosarios), sin imponer una.	Parcial (7 tipos, sin casos de uso)
RF6	Si dos documentos difieren, mostrar siempre la diferencia y cuál es más reciente, sin decidir por el cliente.	Parcial (la diferencia no siempre se muestra)
RF7	Detectar lo ya dicho en esta u otras sesiones: remitir al veredicto anterior o señalar la contradicción.	Parcial (solo repeticiones en la misma sesión)
RF8	Mostrar quién y cuándo dijo lo contradicho.	Parcial (fecha en actas)
RF9	Sugerir una prioridad para cada pedido nuevo, con su razón.	Parcial (solo en la web)
Núcleo: sesión y decisiones		
RF10	Registrar la decisión de cualquier participante, por voz o en la web, y pedir aclaración si hay duda.	PoC
RF11	Distinguir a cada participante, sin necesidad de saber quién es.	PoC
RF12	Al cerrar, devolver a los participantes lo decidido y lo pendiente, y exportar el registro.	PoC
Aplicación web		
RF13	Usarse directamente en la web, con micrófono o con texto escrito.	PoC
RF14	Compartir la sesión entre la web y los conectores, y abrir cada documento en el fragmento citado.	Parcial (solo ejecución local)

ID	El producto debe	Alcance
Conectores de videollamada		
RF15	Entrar a una reunión como participante, a partir del enlace.	PoC
RF16	Transcribir en vivo en español y grabar la sesión completa.	Parcial (no graba)
RF17	Mostrar a todos en la reunión cada veredicto accionable y lo pendiente, en un mensaje corto con acceso al detalle.	Parcial (detalle en la aplicación local)
RF18	Funcionar en más de una plataforma de videollamada.	Parcial (solo Meet)
RF19	Compartir pantalla para mostrar en la reunión lo que ya está en el núcleo: el veredicto y el fragmento citado.	Futuro
Después de la sesión e integraciones		
RF20	Proponer tareas de alto nivel para el equipo (RNF10).	Parcial (solo marca el pedido, en la web)
RF21	Proponer cambios a la documentación según lo decidido (RNF10).	Futuro
RF22	Tomar la documentación de un repositorio, como GitHub.	A explorar
RF23	Publicar las tareas aceptadas en el gestor de tareas, como GitHub.	A explorar

Tabla 4.1: Requisitos funcionales

Los requisitos no funcionales (Tabla 4.2) se agrupan según Sommerville [1, sec. 4.1.2] y siguen el mismo criterio de importancia; RNF1 a RNF5 y RNF10 sostienen el criterio rector (Sección 4.1). RNF11 y RNF12 toman como referencia la ley uruguaya de protección de datos personales, que exige informar previamente a los titulares al recabar sus datos y limita su transferencia internacional y su uso por prestadores de servicios [42, arts. 13, 23 y 30].

ID	Atributo	El producto debe	Alcance
Producto			
RNF1	Inmediatez	Devolver el veredicto mientras el tema sigue en discusión.	PoC
RNF2	Fidelidad	Señalar toda cita ausente de la documentación.	Parcial (el panel del bot no la marca)
RNF3	Trazabilidad	Registrar qué se consultó y cómo se decidió.	PoC
RNF4	Usabilidad	Poder usarse sin instalación ni capacitación previa.	PoC
RNF5	Discreción	Intervenir solo cuando hay algo accionable.	PoC
RNF6	Adaptabilidad	Sumar un tipo de documento, como casos de uso, con su descripción de lectura, sin modificar la validación.	A explorar
RNF7	Portabilidad	Sumar una plataforma sin modificar la validación.	Parcial (solo Meet y web)
RNF8	Confiabilidad	Entrar a la reunión y mostrar veredictos de forma estable.	Parcial (inestable)

ID	Atributo	El producto debe	Alcance
RNF9	Escala y acceso	Atender sesiones simultáneas, con cuentas y permisos por proyecto.	Futuro
Organización			
RNF10	Control humano	No cambiar documentos ni aceptar tareas sin aprobación humana, y permitir deshacer toda decisión [43, G9].	Parcial (sin deshacer en uso directo)
Externos			
RNF11	Transparencia	Identificarse como participante e informar a los presentes que registra lo dicho.	Parcial (se identifica, sin aviso)
RNF12	Cumplimiento	Respetar los términos de cada plataforma y controlar qué reciben y guardan terceros.	Futuro

Tabla 4.2: Requisitos no funcionales

4.3 Diseño

Esta sección diseña la solución conceptual: el producto de la Sección 4.2. La prueba de concepto recorta canales, plataformas y tipos de documento, porque lo restante basta para observar el ciclo de feedback, y mantiene la cita comprobada y la devolución al cliente (Tabla 4.6).

4.3.1 Arquitectura general

La validación solo necesita texto. Por eso se separan la aplicación web y los conectores livianos, que transcriben lo dicho y muestran el resultado; un servidor propio, el núcleo, que ordena los pasos y guarda la sesión; y la base de conocimiento (Figura 4.1). Integrar la validación en cada plataforma obligaría a repetirla; con un solo núcleo, sumar una plataforma es escribir un conector (RF18 y RNF7), en la línea de la arquitectura de puertos y adaptadores [44].

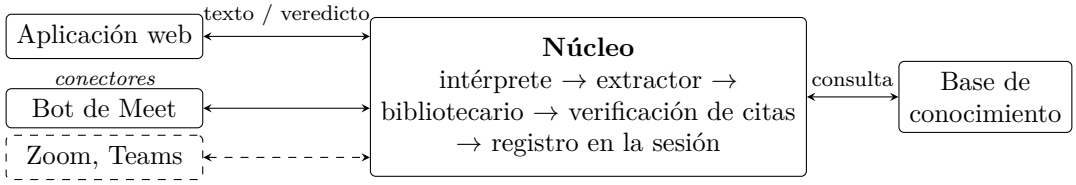


Figura 4.1: Solución conceptual (línea discontinua: conectores sin construir)

4.3.2 Conectores de videollamada

Forma de conexión. La Tabla 4.3 compara las formas de obtener el texto de una reunión. Se elige un bot participante porque no exige configurar la computadora de cada participante y permite responder dentro de la reunión (RF15 y RF17). Se desarrolla en este trabajo, en lugar de contratar un servicio externo de bots, para no depender de un tercero.

Forma	Funcionamiento	Plataformas	Principal limitación
Audio de la computadora	La aplicación web escucha el micrófono, o un dispositivo de audio virtual que redirige el sonido de la reunión [45, 46, 47].	Cualquiera	Configuración manual en cada computadora; el micrófono solo capta lo que suena en la sala.
Navegador	Una extensión lee los subtítulos de la reunión, o la aplicación captura el audio de la pestaña [48, 49].	Meet, Zoom y Teams [50, 51, 52]	La reunión debe abrirse en el navegador; el audio de pestaña solo se captura en Chrome y Edge; la lectura de subtítulos depende del diseño de la página.
Interfaz oficial	La plataforma entrega el audio o la transcripción a una aplicación registrada.	Zoom y Teams; Meet retira la suya el 15/1/2027	Requiere créditos pagos (Zoom), o habilitación del administrador o alojamiento en Azure (Teams).
Bot participante	Un programa entra como un participante más, recibe el audio y, donde la plataforma lo permite, escribe en el chat.	Meet y Teams; en Zoom, sin vía documentada	Condiciones propias de cada plataforma (Tabla 4.4).

Tabla 4.3: Formas de obtener el texto de una reunión

Plataforma. La prueba de concepto usa Google Meet porque un navegador automatizado basta para entrar a la reunión, sin registrar una aplicación, siempre que el organizador invite o admita al bot (RF18). Solo se probó Meet. Google no documenta este uso: sus términos no mencionan bots, pero prohíben eludir sus medidas de protección y permiten suspender la cuenta ante un incumplimiento [53, 54] (RNF12). La Tabla 4.4 resume las condiciones de cada plataforma.

Transcripción. Se usa la de la plataforma donde exista (RF11 y RF16); donde no exista, como en Meet, un servicio de transcripción en vivo (Figura 4.2).

Devolución. El canal depende de la gravedad del veredicto (RF17 y RF19).

Despliegue. Opera un bot por reunión en un servidor propio, porque necesita navegador y conexiones persistentes (RNF8 y RNF9).

Plataforma	Integración posible	Condiciones
Google Meet	Un navegador automatizado entra como participante y toma el audio de la página.	Google retira su interfaz oficial de medios en vivo (Meet Media API) el 15/1/2027 [55]; sus complementos y su API REST no entregan audio en vivo [56, 57]. Si el organizador desactiva las solicitudes de ingreso, Google rechaza a los bots de terceros [58].
Zoom	Una aplicación registrada recibe audio y transcripción en vivo (RTMS, por *Realtime Media Streams*) [59]. Sin vía documentada para un bot participante: su kit de desarrollo de reuniones (Meeting SDK) no admite bots [60].	Zoom detecta e intercepta la automatización de su versión web [61]. RTMS se cobra por minuto en créditos de Zoom (Developer Pack) [62].
Microsoft Teams	Un bot recibe el audio en tiempo real, alojado en la nube de Microsoft (Azure) [63], o una aplicación recibe la transcripción en curso mediante el servicio Work IQ [64].	Teams retiene en la sala de espera a los bots no registrados hasta que el organizador los admite [65]. Work IQ está desactivado por defecto, lo habilita el administrador de la organización que convoca la reunión y solo entrega texto si alguien inició la transcripción [64].

Tabla 4.4: Integración por plataforma, según la documentación oficial consultada en noviembre de 2026

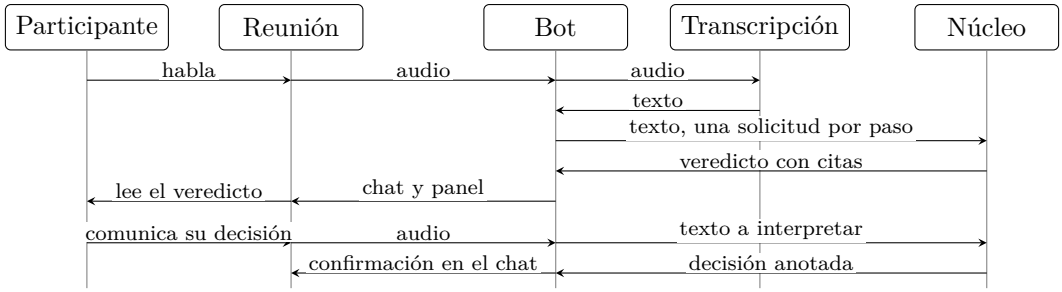


Figura 4.2: Recorrido de una frase y su decisión con el bot de la prueba de concepto

4.3.3 Procesamiento en el núcleo

Cada enunciado recorre los pasos de la Tabla 4.5.

El bibliotecario lee documentos enteros, con tipo y fecha: un acta posterior puede matizar un requisito (RF6), y la búsqueda por similitud habitual en la generación aumentada por recuperación (RAG, por *retrieval-augmented generation*) [66]

Paso	Qué hace	Por qué
Intérprete	Determina si una frase expresa una decisión o repite lo ya dicho (RF7, RF10).	Una decisión breve carece de contenido que validar y, sin este paso, se descartaría.
Extractor	Separa los enunciados que corresponde validar (RF3, RNF5).	La segmentación por pausas fragmenta las ideas.
Bibliotecario	Selecciona los documentos que consulta y emite el veredicto (RF1, RF4, RF9).	Deja constancia de los documentos consultados (RNF3).
Verificación	Comprueba que cada cita figure en los documentos consultados (RF2, RNF2).	Un modelo puede citar fuentes inexistentes [31, sec. 3.5.5]; aquí, toda cita inexistente queda señalada.
Registro	Abrevia el resultado para el chat y el panel, y guarda veredictos y decisiones (RF14).	La aplicación web y los conectores comparten la sesión.

Tabla 4.5: Pasos del procesamiento

traería fragmentos parecidos a lo dicho, no el documento entero. La verificación comprueba que la cita existe, no que respalde el veredicto, lo que no está garantizado [67]: por eso cada documento se abre en el fragmento citado (RF14). El producto también consulta las sesiones anteriores y su grabación (RF7 y RF16).

4.3.4 Base de conocimiento

Como no hay una forma estándar de documentar (Subsección 2.1.1), la base de conocimiento admite todas (RF5, RNF6). Un formato único obligaría a reescribir lo existente, y un modelo común borraría lo que distingue a cada tipo. Por eso cada tipo tiene un espacio y una descripción de cómo se lee (Figura 4.3).

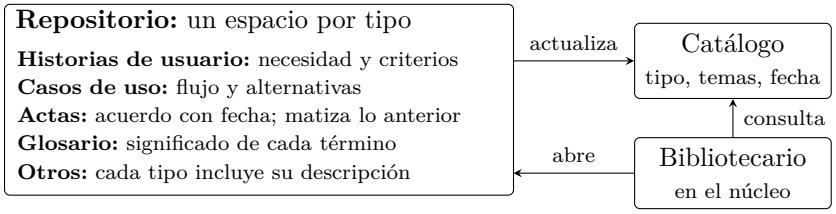


Figura 4.3: Base de conocimiento en la solución conceptual

El repositorio (RF22) guarda el historial de cada documento. De él se obtienen el autor, la fecha y el catálogo actualizado; quién dijo algo surge del documento

o del registro de la sesión (RF8 y RF11). La fecha indica cuál es más reciente, no cuál rige (RF6). Por ejemplo, el cliente dice "el material de referencia no se presta". El requisito lo confirma, pero el acta del 12/5 exceptúa a los docentes de viernes a lunes, y el producto pregunta si sigue vigente.

4.3.5 Conexión con el trabajo del equipo

Se valida en vivo (RNF1), porque allí se cierra el ciclo con el cliente. Los cambios en artefactos del equipo se difieren, como propuestas que una persona acepta (RF20, RF21, RF23, RNF10): escribir en vivo les trasladaría errores de transcripción sin revisar. Si documentación y tareas están en GitHub, un mismo repositorio es entrada y salida del producto (RF22, RF23).

4.3.6 Recorte de la prueba de concepto

Decisión	**Solución conceptual**	**Prueba de concepto**
Orden de pasos	Lo fija el núcleo	Lo fija cada canal; uno nuevo debe repetirlos (RNF7). En uso directo, la web solo emite el veredicto y extrae enunciados si se dicta
Captura	Web: micrófono o texto; reunión: transcripción de la plataforma o de un servicio	Web: micrófono o texto; reunión: transcripción en vivo con hablantes numerados
Conectores	Conector propio por plataforma, en servidor	Bot propio, local, de inicio manual y solo en Meet
Devolución	Web; en la reunión, chat, video y pantalla	Web con detalle; en la reunión, chat con registro y panel en el video del bot
Sesión	Persistente	Archivo local
Documentos	Espacio por tipo con su descripción	Diez, de siete tipos; descripción fija
Fuente	Repositorio con catálogo actualizado	En la aplicación, catálogo manual
Fecha y autor	Del historial y del registro; diferencia siempre visible	Fecha sin autor; rige la más reciente; diferencia no siempre visible
Salidas	Tareas y cambios aceptados	Solo registro exportado

Tabla 4.6: Solución conceptual y prueba de concepto

4.4 Implementación

Comentario: Sección pendiente: arquitectura implementada, qué requisitos cubre la prueba de concepto y cómo se ve. Se redacta después de profundizar el prototipo.

4.5 Validación

Comentario: Sección pendiente. Encuadre acordado: reuniones simuladas de unos 15 minutos sobre el caso de estudio ficticio, con personas invitadas, y una encuesta breve sobre la interacción con el bot.

5 Conclusiones

Comentario: La conclusión debe resumir la contribución del trabajo y enfatizar el resultado principal. También puede abordar la aplicación del resultado y las cuestiones abiertas, y dar futuras direcciones de investigación. Por lo general se estructura con un breve resumen del trabajo y sus objetivos, principales resultados obtenidos, aplicación de los resultados, limitaciones del trabajo y líneas de trabajo futuro

6 Referencias Bibliográficas

[1] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016, global Edition (adaptada de la 10.ª edición estadounidense, ISBN 978-0-13-394303-0).

[2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. McGraw-Hill Education, 2020.

[3] K. Schwaber and J. Sutherland, "The 2020 scrum guide," Nov. 2020, licencia Creative Commons BY-SA 4.0. [Online]. Available: https://scrumguides.org/scrum-guide.html

[4] M. Dumas, M. La Rosa, J. Mendling, and H. A. Reijers, *Fundamentals of Business Process Management*, 2nd ed. Springer, 2018.

[5] J. Sauvola, S. Tarkoma, M. Klemettinen, J. Riekki, and D. Doermann, "Future of software development with generative AI," *Automated Software Engineering*, vol. 31, no. 26, 2024. [Online]. Available: https://doi.org/10.1007/s10515-024-00426-z

[6] A. Nguyen-Duc, B. Cabrero-Daniel, A. Przybylek, C. Arora, D. Khanna, T. Herda, U. Rafiq, J. Melegati, E. Guerra, K.-K. Kemell, M. Saari, Z. Zhang, H. Le, T. Quan, and P. Abrahamsson, "Generative artificial intelligence for software engineering—a research agenda," *Software: Practice and Experience*, vol. 55, no. 11, pp. 1806–1843, 2025. [Online]. Available: https://doi.org/10.1002/spe.70005

[7] A. Fawzy, A. Tahir, and K. Blincoe, "From prompting to verification: How experience shapes vibe coding practices," 2026, preprint (literatura gris). [Online]. Available: https://arxiv.org/abs/2605.24521

[8] Y. Virk and D. Liu, "Non-programmers assessing AI-generated code: A case study of business users analyzing data," in *IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, 2025. [Online]. Available: https://arxiv.org/abs/2508.06484

[9] H. Washizaki, Ed., *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0.* IEEE Computer Society, 2024. [Online]. Available: https://www.swebok.org

[10] M. Glinz, H. van Loenhoud, S. Staal, and S. Bühne, *Foundation Level Handbook, versión 1.1.1*, International Requirements Engineering Board (IREB), 2024. [Online]. Available: https://gasq.org/files/content/gasq/downloads/certification/IREB/cpre_foundationlevel_handbook_en_v1.1.1.pdf

[11] S. Wagner, D. Méndez Fernández, M. Felderer, A. Vetrò, M. Kalinowski, R. Wieringa, D. Pfahl, T. Conte, M.-T. Christiansson, D. Greer, C. Lassenius, T. Männistö, M. Nayebi, M. Oivo, B. Penzenstadler, R. Prikladnicki, G. Ruhe, A. Schekelmann, S. Sen, R. Spínola, A. Tuzcu, J. L. de la Vara, and D. Winkler, "Status quo in requirements engineering: A theory and a global family of surveys," *ACM Transactions on Software Engineering and Methodology*, vol. 28, no. 2, pp. 1–48, 2019.

[12] X. Franch, C. Palomares, C. Quer, P. Chatzipetrou, and T. Gorschek, "The state-of-practice in requirements specification: an extended interview study at 12 companies," *Requirements Engineering*, vol. 28, no. 3, pp. 377–409, 2023.

[13] P. Rook, "Controlling software projects," *Software Engineering Journal*, vol. 1, no. 1, pp. 7–16, Jan. 1986. [Online]. Available: https://doi.org/10.1049/sej.1986.0003

[14] OpenAI, "GPT-4 technical report," 2023, informe técnico. Preprint (literatura gris). [Online]. Available: https://arxiv.org/abs/2303.08774

[15] N. V. Malladi and K. Sudheer Reddy, "Generative AI in agile software development: A comprehensive survey," in *2025 7th International Conference on Innovative Data Communication Technologies and Application (ICIDCA)*. IEEE, 2025. [Online]. Available: https://doi.org/10.1109/ICIDCA66325.2025.11280486

[16] H. Cheng, J. H. Husen, Y. Lu, T. Racharak, N. Yoshioka, N. Ubayashi, and H. Washizaki, "Generative AI for requirements engineering: A systematic literature review," *Software: Practice and Experience*, vol. 56, no. 2, pp. 141–170, 2026.

[17] P. Vasudevan and S. Reddivari, "The role of generative AI models in requirements engineering: A systematic literature review," in *Proceedings of the 2025 ACM Southeast Conference*, 2025, pp. 188–194.

[18] J. Lang and M. Fischer, "Generative artificial intelligence for requirements engineering in software development–analysis of the state-of-the-art," in *HCI International 2025–Late Breaking Papers: 27th International Conference on Human-Computer Interaction, HCII 2025, Gothenburg, Sweden, June 22–27, 2025, Proceedings, Part XVI*. Springer, 2026, pp. 175–185.

[19] M. A. Abbasi, P. Ihantola, T. Mikkonen, and N. Mäkitalo, "Towards human-AI synergy in requirements engineering: A framework and preliminary study," in *2025 Sixth International Conference on Intelligent Data Science Technologies and Applications (IDSTA)*. IEEE, 2025, pp. 81–88.

[20] B. Cabrero-Daniel, T. Herda, V. Pichler, and M. Eder, "Exploring human-AI collaboration in agile: Customised LLM meeting assistants," in *International Conference on Agile Software Development*. Springer, 2024, pp. 163–178.

[21] F. Kretzer, K. Kolthoff, C. Bartelt, S. P. Ponzetto, and A. Maedche, "Closing the loop between user stories and GUI prototypes: an LLM-based assistant for cross-functional integration in software development," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 2025, pp. 1–19.

[22] P. Lindenberg, I. Kumara, J. Owotogbe, W.-J. van den Heuvel, and D. A. Tamburri, "Business process discovery through agentic generative AI," in *Service-Oriented Computing (ICSOC 2025)*, ser. Lecture Notes in Computer Science, vol. 16321. Springer Nature Singapore, 2025, pp. 253–267. [Online]. Available: https://doi.org/10.1007/978-981-95-5015-9_19

[23] M. Mircea, E. Gevrek, E. Schmid, and K. Schneider, "Supporting stakeholder requirements expression with LLM revisions: An empirical evaluation," in *Requirements Engineering: Foundation for Software Quality (REFSQ 2026)*, ser. Lecture Notes in Computer Science. Springer Nature Switzerland, 2026, pp. 303–319. [Online]. Available: https://doi.org/10.1007/978-3-032-21423-2_21

[24] D. Robinson, C. Cabrera, A. D. Gordon, N. D. Lawrence, and L. Mennen, "Requirements are all you need: The final frontier for end-user software engineering," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 1–22, 2025.

[25] H. Alabsi, S. Srinivasan, R. Obeidat, N. Lakew, M. Andrew, and A. Zenebe, "An empirical evaluation of LLM-assisted sketch-based requirements elicitation and prototyping," 2026, preprint, Research Square, sin revisión por pares. [Online]. Available: https://doi.org/10.21203/rs.3.rs-8475358/v1

[26] N. Busany, E. Hadar, H. Hadad, G. Rosenblum, Z. Maszlanka, O. Akhigbe, and D. Amyot, "Automating business intelligence requirements with generative AI and semantic search," 2024, preprint (literatura gris). [Online]. Available: https://arxiv.org/abs/2412.07668

[27] D. Damian, B. Ghimire, Z. S. Li, and K. Devathasan, "REConnect: Participatory RE that matters," 2025, preprint (literatura gris). [Online]. Available: https://arxiv.org/abs/2509.01006v1

[28] W. Geyer, J. He, D. Sarkar, M. Brachman, C. Hammond, J. Heins, Z. Ashktorab, C. Rosemberg, and C. Hill, "A case study investigating the role of generative AI in quality evaluations of epics in agile software development," 2025, preprint (literatura gris); también publicado en ACM Digital Library, jun 2025. [Online]. Available: https://arxiv.org/abs/2505.07664

[29] U. B. Torun, M. T. Demircan, M. F. Gön, and E. Tüzün, "Past, present, and future of bug tracking in the generative AI era," 2025, preprint (literatura gris). [Online]. Available: https://arxiv.org/abs/2510.08005

[30] B. Violino, "CIOs are enlisting business users to vibe code their own apps," CIO, May 2026, publicado el 26-may-2026. Prensa profesional, sin revisión por pares (literatura gris). [Online]. Available: https://www.cio.com/article/4176062/cios-are-enlisting-business-users-to-vibe-code-their-own-apps.html

[31] N. Sharma, Z. Zhang, D. Lee, N. Krishnan, G.-J. Ren, Z. Xiao, and Y. Li, "Feedback by design: Understanding and overcoming user feedback barriers in conversational agents," in *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, 2026. [Online]. Available: https://arxiv.org/abs/2602.01405

[32] H. Cornide-Reyes, D. Monsalves, E. Durán, F. Silva-Aravena, and J. Morales, "Generative artificial intelligence in agile software development processes: A literature review focused on user eXperience," in *Social Computing and Social Media*, ser. Lecture Notes in Computer Science. Springer Nature Switzerland, 2025, pp. 228–246. [Online]. Available: https://doi.org/10.1007/978-3-031-93536-7_16

[33] M. Pirozzi, "The stakeholder perspective in the generative artificial intelligence scenario and the AI-stakeholders," *PM World Journal*, vol. XIII, no. VIII, Aug. 2024, featured Paper. Revista profesional, sin revisión por pares (literatura gris). [Online]. Available: https://pmworldlibrary.net/wp-content/uploads/2024/08/pmwj144-Aug2024-Pirozzi-Stakeholder-Perspective-in-Generative-AI-Scenario-and-AI-Stakeholders.pdf

[34] Kraftful, “Kraftful,” https://www.ycombinator.com/companies/kraftful.

[35] Versive, “Versive,” https://www.ycombinator.com/companies/versive.

[36] Emergent, “Emergent,” https://www.ycombinator.com/companies/industry/generative-ai/san-francisco.

[37] Heizen, “PM agent,” https://www.heizen.work.

[38] Cognition AI, “Devin,” https://devin.ai.

[39] Codegen, “Codegen,” https://codegen.com.

[40] Tusk, “Tusk: AI coding agent,” https://www.ycombinator.com/launches/KUe-tusk-ai-coding-agent-that-completes-your-chores.

[41] Sweep, “Sweep,” https://www.ycombinator.com/companies/sweep.

[42] Uruguay, “Ley N.ž 18.331: Ley de protección de datos personales,” 2008, consultada el 6/11/2026. [Online]. Available: https://www.impo.com.uy/bases/leyes/18331-2008

[43] S. Amershi, D. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. Iqbal, P. N. Bennett, K. Inkpen, J. Teevan, R. Kikin-Gil, and E. Horvitz, “Guidelines for human-AI interaction,” in *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, 2019, pp. 1–13.

[44] A. Cockburn, “The hexagonal (ports & adapters) architecture,” HaT Technical Report 2005.02, v0.9, 2005, consultado el 6/11/2026. [Online]. Available: https://alistair.cockburn.us/hexagonal-architecture/

[45] MDN Web Docs, “MediaDevices: getUserMedia() method,” 2026, consultado el 6/11/2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia

[46] Existential Audio, “BlackHole,” 2026, consultado el 6/11/2026. [Online]. Available: https://github.com/ExistentialAudio/BlackHole

[47] VB-Audio, “VB-Audio virtual apps,” 2026, consultado el 6/11/2026. [Online]. Available: https://vb-audio.com/Cable/

[48] MDN Web Docs, “MediaDevices: getDisplayMedia() method,” 2026, consultado el 6/11/2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getDisplayMedia

[49] W3C, “Screen capture,” 2026, consultado el 6/11/2026. [Online]. Available: https://www.w3.org/TR/screen-capture/

[50] Google, “Learn what requirements you need to use Google Meet,” 2026, consultado el 6/11/2026. [Online]. Available: https://support.google.com/meet/answer/7317473

[51] Zoom, “Viewing captions in a meeting or webinar,” 2026, consultado el 6/11/2026. [Online]. Available: https://support.zoom.com/hc/en/article?id=zm_kb&sysparm_article=KB0059762

[52] Microsoft, “Teams for web - overview and prerequisites,” 2026, consultado el 6/11/2026. [Online]. Available: https://learn.microsoft.com/en-us/microsoftteams/teams-client-web

[53] Google, “Google terms of service,” 2026, consultado el 6/11/2026. [Online]. Available: https://policies.google.com/terms

[54] ——, “Google Meet acceptable use policy,” 2025, consultado el 6/11/2026. [Online]. Available: https://support.google.com/meet/answer/9847091

[55] ——, “Google Meet API release notes,” 2026, consultado el 6/11/2026. [Online]. Available: https://developers.google.com/workspace/meet/release-notes

[56] ——, “Resource summary for meet.addons.js,” 2026, consultado el 6/11/2026. [Online]. Available: https://developers.google.com/workspace/meet/add-ons/reference/websdk/addon_sdk

[57] ——, “Work with artifacts,” 2026, consultado el 6/11/2026. [Online]. Available: https://developers.google.com/workspace/meet/api/guides/artifacts

[58] ——, “Tips to control meeting access and participation,” 2026, consultado el 6/11/2026. [Online]. Available: https://support.google.com/a/users/answer/11989526

[59] Zoom, “Overview of RTMS for meetings and webinars,” 2026, consultado el 6/11/2026. [Online]. Available: https://developers.zoom.us/docs/rtms/meetings/

[60] ——, “Zoom Meeting SDK for web,” 2026, consultado el 6/11/2026. [Online]. Available: https://developers.zoom.us/docs/meeting-sdk/web/

[61] ——, “Blocking bots from joining meetings and webinars,” 2026, consultado el 6/11/2026. [Online]. Available: https://support.zoom.com/hc/en/article?id=zm_kb&sysparm_article=KB0088381

[62] ——, “Developer pricing,” 2026, consultado el 6/11/2026. [Online]. Available: https://zoom.us/pricing/developer

[63] Microsoft, “Build application-hosted media bots,” 2026, consultado el 6/11/2026. [Online]. Available: https://learn.microsoft.com/en-us/microsoftteams/platform/bots/calls-and-meetings/requirements-considerations-application-hosted-media-bots

[64] ——, “Manage transcript API access for Teams meetings,” 2026, consultado el 6/11/2026. [Online]. Available: https://learn.microsoft.com/en-us/microsoftteams/meeting-transcript-api-access

[65] ——, “Introducing smarter bot protection in Microsoft Teams meetings,” 2026, consultado el 6/11/2026. [Online]. Available: https://techcommunity.microsoft.com/blog/microsoftteamsblog/introducing-smarter-bot-protection-in-microsoft-teams-meetings/4531375

[66] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf

[67] N. F. Liu, T. Zhang, and P. Liang, “Evaluating verifiability in generative search engines,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 7001–7025.

7 Anexos

7.1 Titulo Anexo 1

7.2 Titulo Anexo 2